

Applications of Hashing (With Copious Tangents...)

Design and Analysis of Algorithms

Brian C. Dean

**School of Computing
Clemson University**

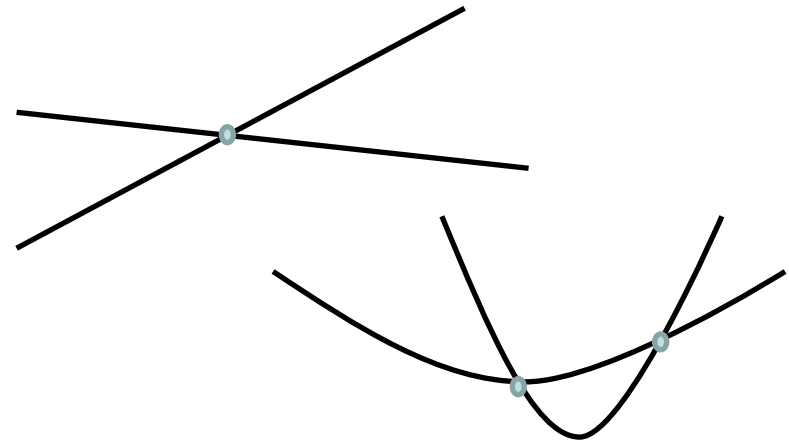


Recall: Applications of Hashing Large Objects

- Fast string matching
- Testing if files in different locations (e.g., cloud backup) are the same.
- Detecting tampering with critical files.
- Testing if files transferred over noise communication channel without corruption (appending a hash “checksum”)

Recall: Collision Probability of Polynomial Hash Functions

- Hash $A[0 \dots N-1]$ to an integer in $0 \dots M-1$ by evaluating $(A[0] + A[1]x + A[2]x^2 + A[3]x^3 + \dots + A[N-1]x^{N-1}) \% M$ at a random point x in $\{0, 1, 2, \dots, M-1\}$.
- Key insight: At most $N-1$ “bad” values of x that cause $A[] \neq B[]$ to collide:
 - Two lines can agree in at most 1 point.
(equivalent to “two points determine a line”)
 - Two quadratics can agree in at most 2 points.
(equivalent to the quadratic representing their difference having at most 2 roots)
 - In general, degree- $(N-1)$ polynomials can agree in at most $N-1$ points (and this is still true if we do integer arithmetic modulo a prime)
 - So probability of collision is at most $(N-1) / M \leq N / M$

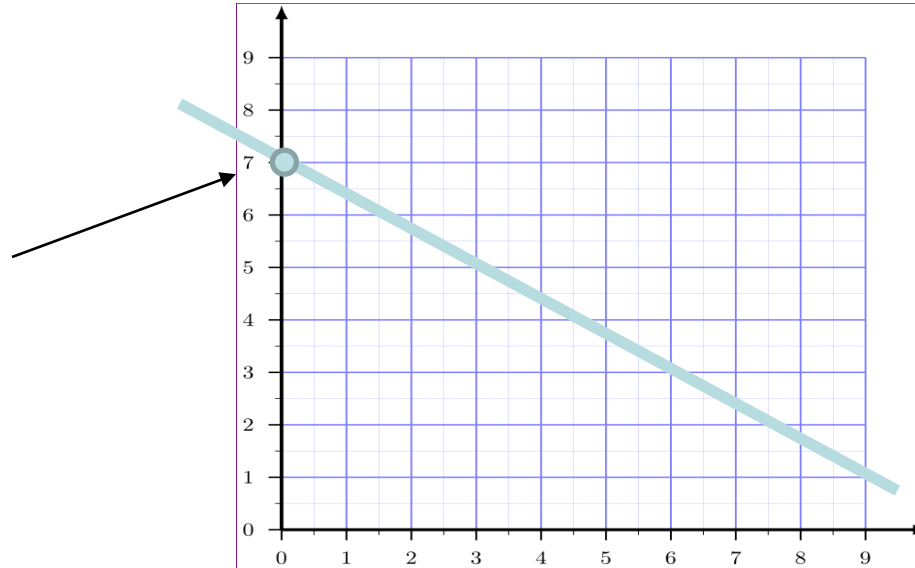


Interesting Challenge...

- Can I distribute a secret number among all N people in class so that:
 - Any two people, working together, can discover the secret.
 - Any one person, by themselves, knows effectively nothing about the secret.
- **Example:** In Russia in 1992, any two of the following three parties had to agree to order the arming of nuclear weapons: President Boris Yeltsin, Defense Minister Yevgeni Shaposhnikov, and the Defense Ministry chief of staff.

Secret Sharing

Secret = 7



- **Easy Solution:** Make the secret the y-intercept of a line, and tell each person one point on the line.
- What if we need a group of 3+ to determine the secret, while any group of <3 should know nothing about it? Use a quadratic!

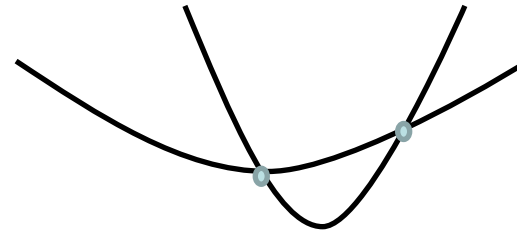
Aside²: Error-Correcting Codes

- Two quadratics can agree in at most 2 points.

$$p_A(x) = A[0] + A[1]x + A[2]x^2$$

$$p_B(x) = B[0] + B[1]x + B[2]x^2$$

(equivalent to “3 points determine a quadratic”)



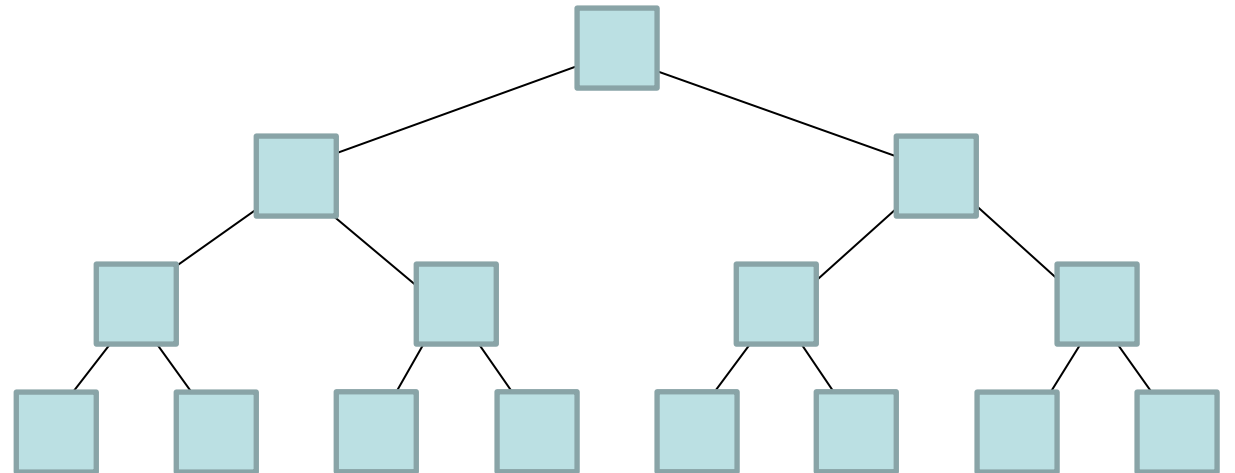
- So we can transmit $A[0..2]$ (three numbers) across a communication channel in a roundabout way by sending the value of $p_A(x)$ at 3 different values of x .
- What if we transmit $p_A(x)$ at 4 or 5 different values of x ?

Applications in Security

- Store (ideally “salted”) passwords as hashes, rather than “in the clear”.
- Digital signatures: Alice can sign a message by encrypting a message hash with her private key (so only her public key can decrypt).
- “Cryptographic” hash functions (e.g., SHA1) designed to be “one way” / hard to invert – easy to compute $h(x)$ but hard to tamper with x while preserving its hash.

Merkle Trees

- Binary search tree with hash of root from trusted source.
- Each node has a hash that can authenticate its children.



Applications in Cryptocurrencies:

Bitcoin Mining

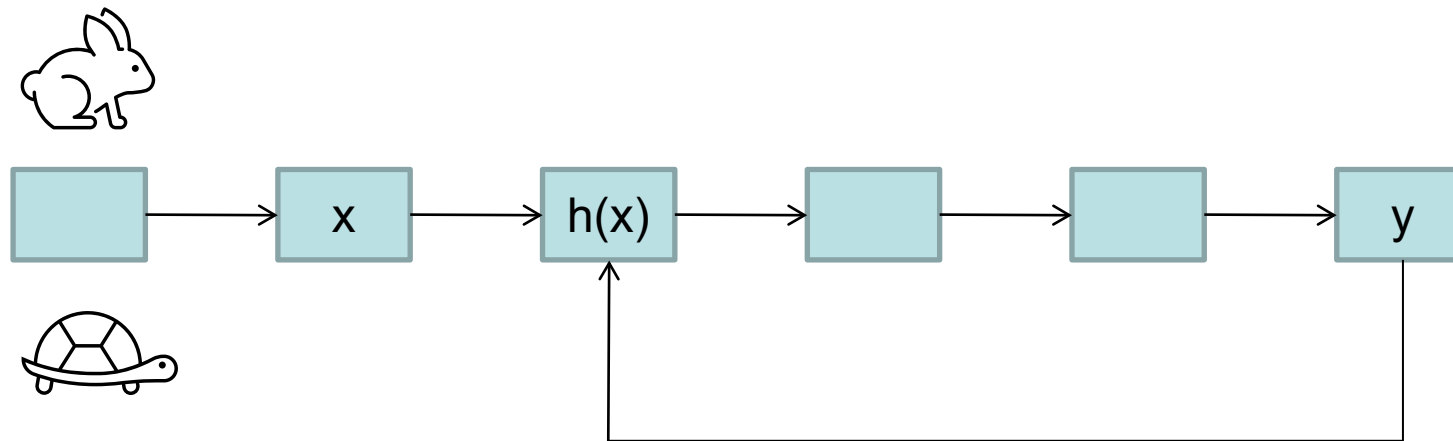
- Intentionally make it hard for one actor to be able to exert too much authorship control in distributed ledger of all transactions.
- Can only publish a block on the “blockchain” if you solve a hard problem.
 - E.g., find x such that $h(\text{“problem17,x”}) \bmod 1,000,000,000 = 0$.

How Would You Find a Pair of Colliding Keys, By the Way?

- Suppose $h(x)$ produces an 80-bit integer.
- We'd like to find x and y such that $h(x) = h(y)$.
- Test random values of $k_1, k_2, \dots$ until we find a collision.
- By the birthday paradox, expect to find a collision after testing about $2^{40} = \sqrt{2^{80}}$ of all possible keys.
- Difficulty: we need to store the hashes of these keys, so we need 2^{40} space as well as time it would seem...

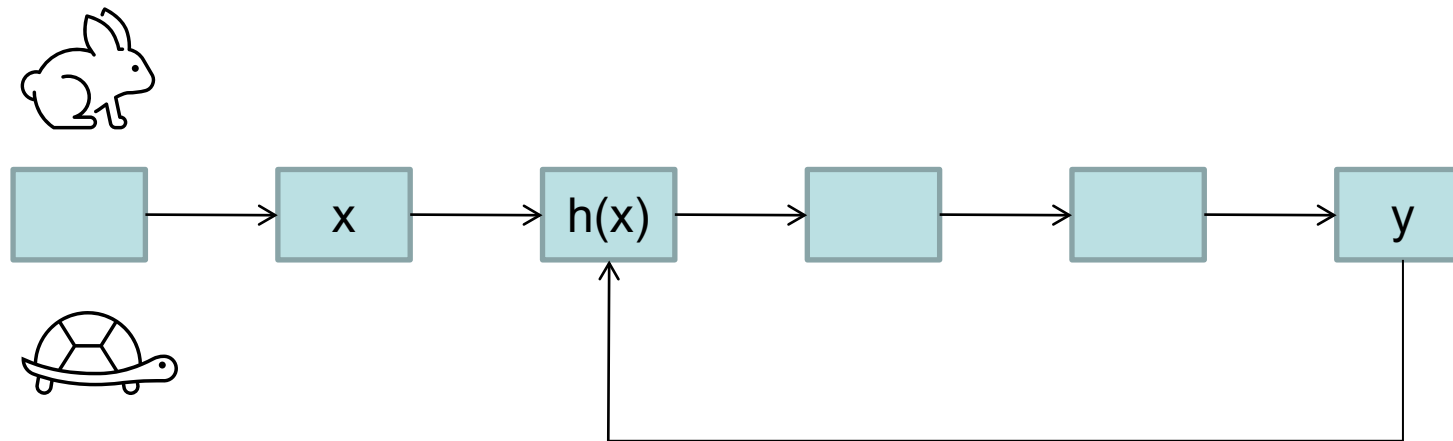
Save Space Using the Slow / Fast Pointer Trick!

- Suppose $h(x)$ produces an 80-bit integer.
- We'd like to find x and y such that $h(x) = h(y)$.
- Test random values of $k_1, k_2, \dots$ until we find a collision.
- Expect to find a collision after testing about $2^{40} = \sqrt{2^{80}}$ keys.



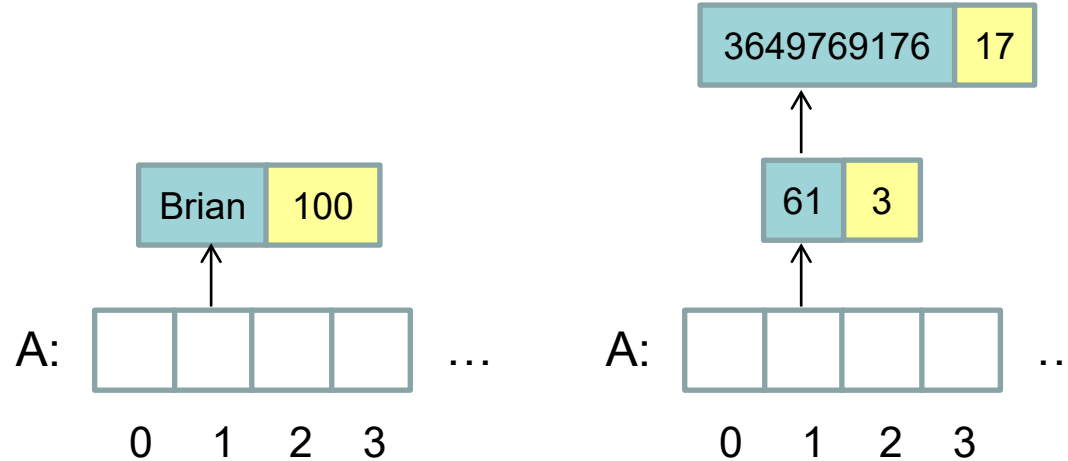
Aside²: How Can You Test if a Program in a VM is Looping Infinitely?

- Maintain a hash of contents of all memory.
- If later on we see the same hash, suspect we might be stuck in a loop (use hash size well over runtime squared!)
- Run two program instances, one twice as fast as the other.



Recall: Maps and Sparse Arrays

- Hash tables ideal for emulating arrays, particularly those only sparsely filled
 - E.g., $A[\text{"Brian"}] = 100$ or $A[3649769176] = 17$; $A[61] = 3$;



- Similar performance to using an actual array in terms of time and space performance! – $O(1)$ expected time per access
- Can use array “index” that is a more interesting object – e.g., a string, an image, a file, ...

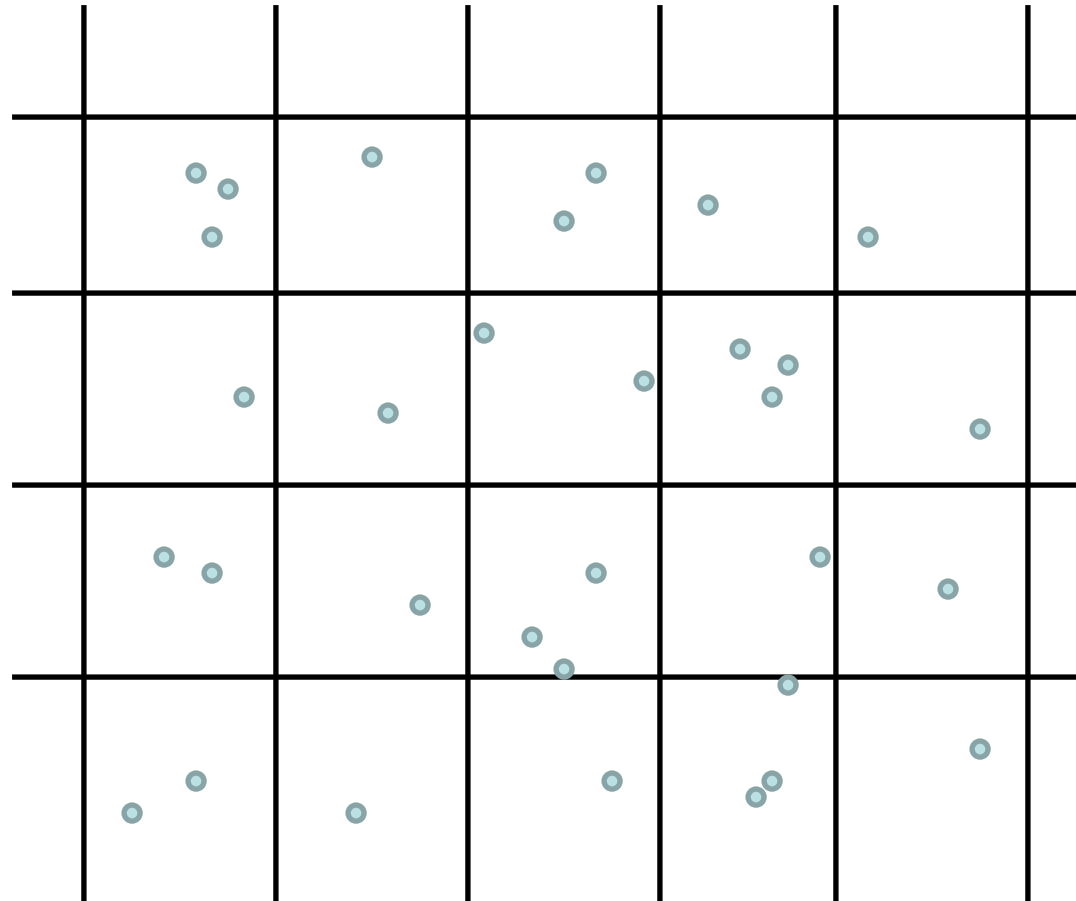
Recall: Problem Decomposition

Given N points uniformly spread across the unit square in 2D, find the closest pair.

Use $\sqrt{N} \times \sqrt{N}$ grid to make N “bins”

So N balls $\rightarrow N$ bins

$E[\# \text{ balls} / \text{bin}] = 1$

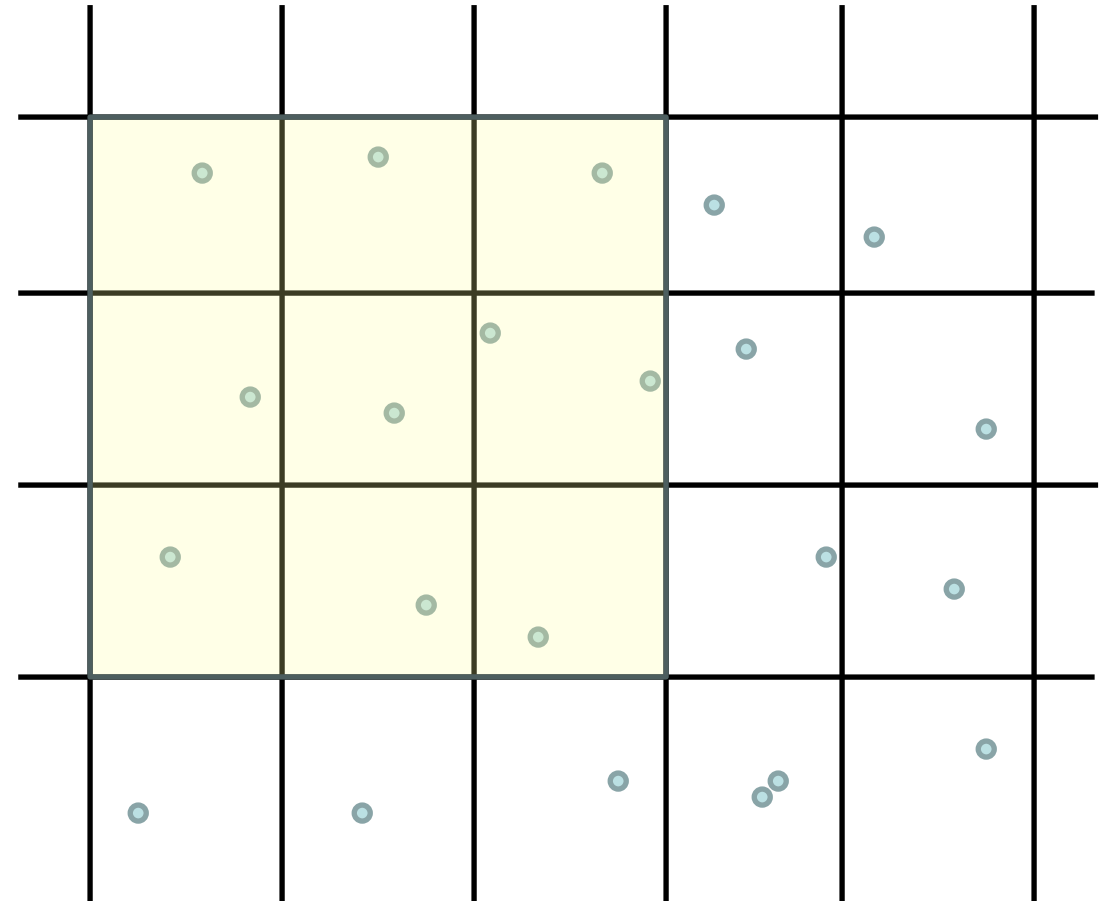


Exploiting Collisions

We are using collisions here in a beneficial way!

Other examples:

- Anagram detection (our coding discussion)
- Locality-sensitive hashing (our enrichment lecture)



Closest Pair of Points Revisited

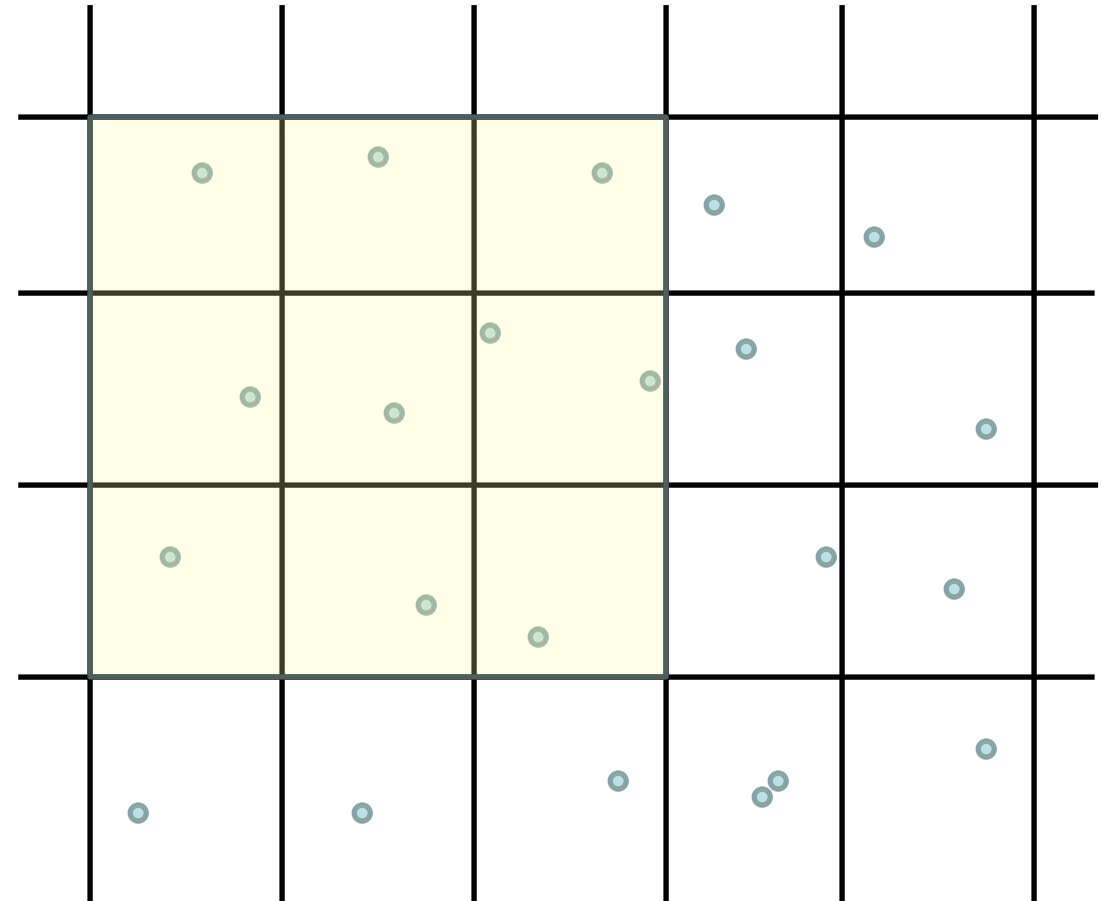
Given N points ~~uniformly spread~~ across the unit square in 2D, is there a pair closer than Δ ?

Use grid with cell size $\Delta \times \Delta$

5 points in the same cell? Can stop, since two are definitely closer than Δ .

If not, just compare every point to the ≤ 4 points in its 9 cell local neighborhood.

$O(1)$ time per point / $O(n)$ total.



Closest Pair of Points Revisited

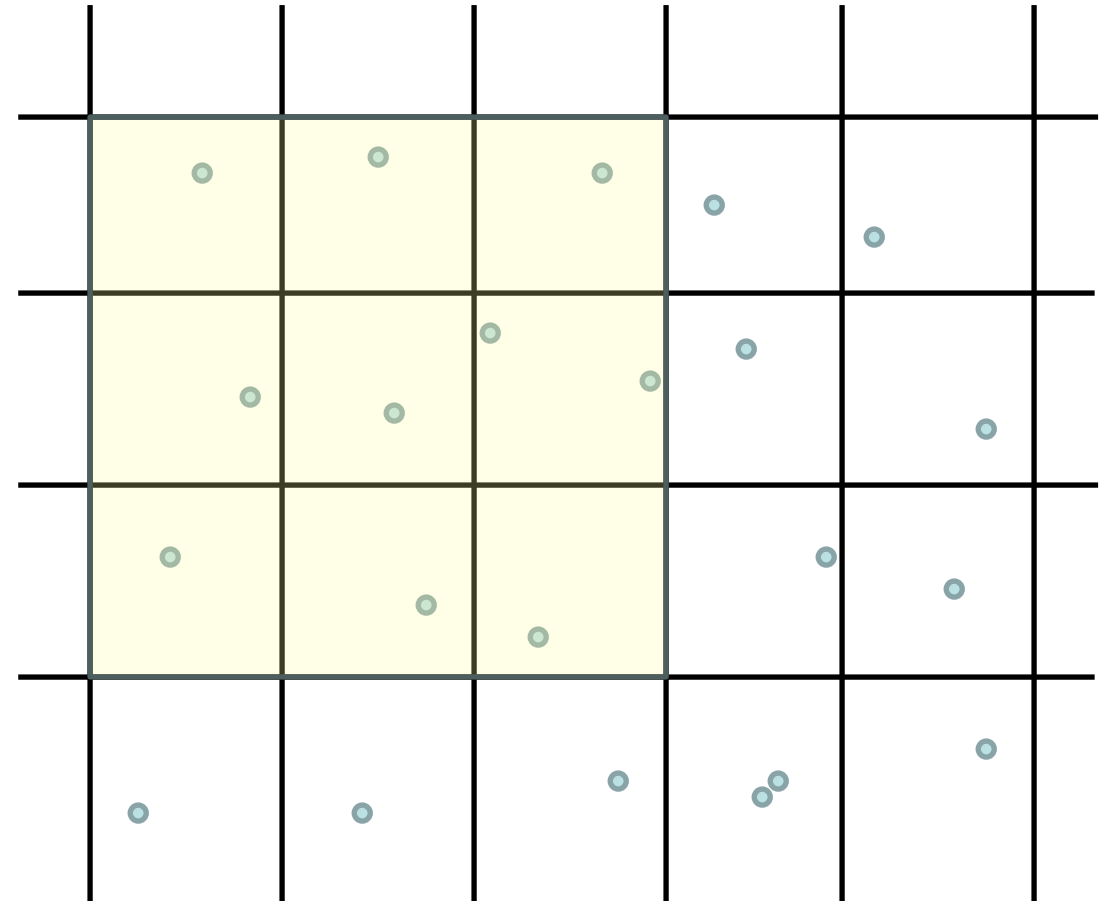
Given N points ~~uniformly spread~~ across the unit square in 2D, is there a pair closer than Δ ?

Use grid with cell size $\Delta \times \Delta$

But wait, the grid could be huge! How can we store such a large (but sparsely filled) 2D array?

Emulate using a hash map!

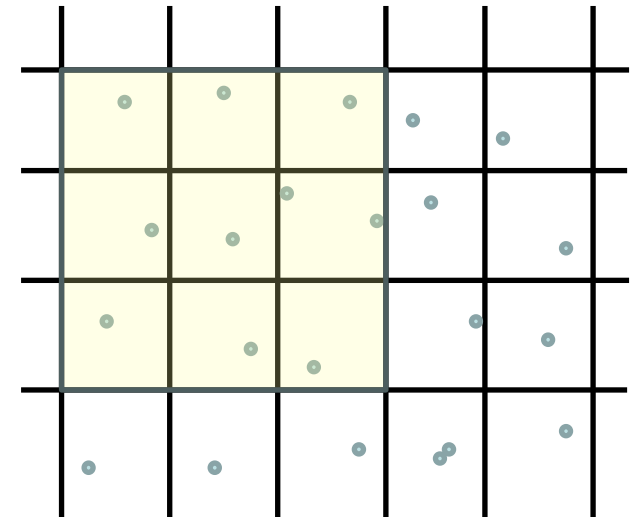
So $O(n)$ expected time in total.



Closest Pair of Points Revisited

Given N points ~~uniformly spread~~ across the unit square in 2D, what is the closest pair of points?

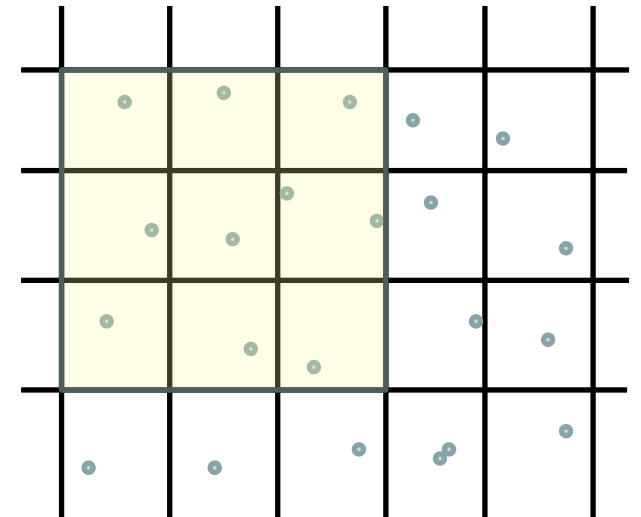
- 3 elegant ways to extend our “close pair” algorithm so it finds the closest pair in $O(n)$ expected time.
 - Random sampling: sample $\sqrt{n}$ points, let Δ be their closest pair distance, run prior algorithm.
 - Randomized incremental construction
 - Divide and conquer



Closest Pair of Points Revisited

Randomized incremental construction:

- Add points in random order to structure, starting with Δ = distance between first two points
- If encounter closer pair than Δ (closest so far), then rebuild entire structure with new grid size Δ' .
- **Pr**[rebuild on adding kth point] = $2 / k$.
(since kth point equally likely to be one of the 2 defining the closest pair of points 1..k)
- Time for rebuilding at kth point = $O(k)$,
so **E**[time for all rebuilds] = $O(n)$.



Closest Pair of Points Revisited

Epic divide and conquer approach:

- Divide points into 10 sets of size $n / 10$.
- Make $\binom{10}{2} = 45$ subproblems, each involving a pair of sets.
- Closest pair will be found if we recurse on all of them.
- $T(n) = 45 T(n / 5) + O(n)$ --- uh oh!
- Solve subproblems in random order, only recursing when our $O(n)$ expected time close pair algorithm says its worthwhile.
- $T(n) = (\sim \ln 45) T(n / 5) + O(n)$ ☺

